

LEARNQUEST

LINUX LABS DOCUMENTATION

MODULE 4

Searching, Viewing, Editing & Processing Text

Command-Line Practice Documentation

Linux Fundamentals Coursework

July 2026

Introduction

The Linux command line is the foundation for navigating, managing, and troubleshooting any Linux-based system. This module brings together practical, hands-on commands used to search through files, view and edit their contents, gather quick statistics, and transform text — the everyday building blocks every Linux user, administrator, or developer relies on. Rather than presenting commands as a single unordered list, this documentation groups them by purpose, pairs each command with a clear explanation, and includes terminal screenshots so the syntax and output can be seen exactly as they appear in practice.

Learning Objectives

- Search and filter text inside files using pattern-matching with `grep`, including anchors, negation, and extended regular expressions.
- View, create, overwrite, and append file content using `cat` and `ls`, and understand the difference between the `>`, `>>`, and `<<` redirection operators.
- Generate quick file statistics — line, word, and character counts — using `wc`, and combine it with other commands through piping.
- Navigate and edit files directly in the terminal using the `nano` and `vi` text editors.
- Apply `sed` to perform stream-based find-and-replace operations and redirect the results into new files.
- Understand how pipes (`|`) and redirection operators chain simple commands together into more powerful workflows.

Learning Path

1. Start with Searching & Filtering — learn to locate patterns inside files with `grep` before manipulating them.
2. Move to Viewing & Writing Files — practice reading, overwriting, and appending content with `cat` and `ls`.
3. Learn File Statistics — use `wc` to measure file size and combine it with pipes for filtered output.
4. Explore Text Editors — get comfortable editing files interactively using `nano` and `vi`.
5. Finish with Text Processing — use `sed` to automate find-and-replace edits across files.

Command Reference by Category

Each category below lists the relevant commands with a short explanation, followed by the terminal screenshot(s) demonstrating that command in action.

1. Searching & Filtering — `grep`

Locating patterns and lines of text inside files.

Command	Description
<code>grep daemon.*nologin /etc/passwd</code>	Searches <code>/etc/passwd</code> for the line that ends with “nologin”, matching anything before it starting with “daemon”.
<code>grep ^root /etc/passwd</code>	Searches <code>/etc/passwd</code> for the line that starts with “root”.

Command	Description
<code>grep -v nologin\$ /etc/passwd</code>	Displays only the lines that do NOT end with “nologin” (inverted match with -v).
<code>grep -E "root coder" /etc/passwd</code>	Uses extended regular expressions (-E) to find lines containing either “root” or “coder”.

```

coursera
File Edit Selection View Go Run Terminal Help code-server
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
coder@fb3cb94990c3:~$ cat/etc/passwd
bash: cat/etc/passwd: No such file or directory
coder@fb3cb94990c3:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534:./nonexistent:/usr/sbin/nologin
sshd:x:101:65534:./run/ssh:/usr/sbin/nologin
coder:x:999:999:./home/coder:/bin/bash
coder@fb3cb94990c3:~$ grep daemon .*nologin /etc/passwd
grep: .*nologin: No such file or directory
coder@fb3cb94990c3:~$ grep daemon .*nologin /etc/passwd

```

Figure 1.1 — `grep` searching `/etc/passwd` by pattern and anchor.

```

coursera
File Edit Selection View Go Run Terminal Help code-server
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
grep: .*nologin: No such file or directory
coder@fb3cb94990c3:~$ grep daemon.*nologin /etc/passwd
coder@fb3cb94990c3:~$ grep daemon.*nologin /etc/passwd
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
coder@fb3cb94990c3:~$ grep ^root etc/passwd
grep: etc/passwd: No such file or directory
coder@fb3cb94990c3:~$ grep ^root /etc/passwd
root:x:0:0:root:/root:/bin/bash
coder@fb3cb94990c3:~$ grep -v nologin$ /etc/passwd
root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
coder:x:999:999:./home/coder:/bin/bash
coder@fb3cb94990c3:~$ grep "root|coder" /etc/passwd
coder@fb3cb94990c3:~$ grep root -E"root|coder" /etc/passwd
grep: invalid option -- 't'
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
coder@fb3cb94990c3:~$ grep -E"root|coder" /etc/passwd
grep: invalid option -- 't'
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
coder@fb3cb94990c3:~$ grep -E "root|coder" /etc/passwd
root:x:0:0:root:/root:/bin/bash
coder:x:999:999:./home/coder:/bin/bash
coder@fb3cb94990c3:~$ ls -l
total 1
drwxr-xr-x 2 coder coder 2 Jul 6 2021 project

```

Figure 1.2 — `grep -v` excluding lines that match a pattern.

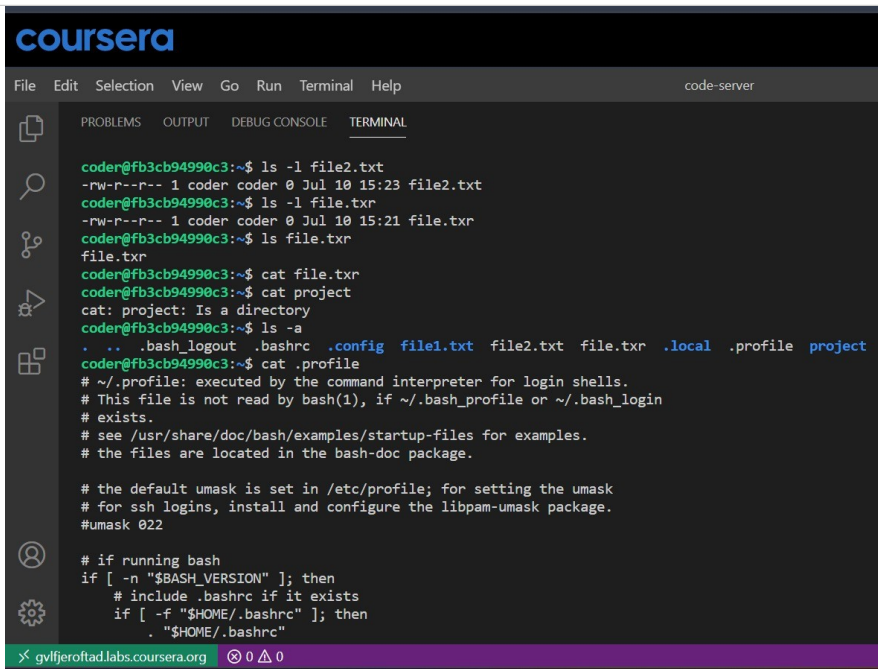
```
drwxr-xr-x 2 coder coder 2 Jul  6  2021 project
coder@fb3cb94990c3:~$ mkdir file1.txt
coder@fb3cb94990c3:~$ ls -l
total 1
drwxr-xr-x 2 coder coder 2 Jul 10 15:21 file1.txt
drwxr-xr-x 2 coder coder 2 Jul  6  2021 project
coder@fb3cb94990c3:~$ touch file.txr
coder@fb3cb94990c3:~$ ls -l
total 2
drwxr-xr-x 2 coder coder 2 Jul 10 15:21 file1.txt
-rw-r--r-- 1 coder coder 0 Jul 10 15:21 file.txr
drwxr-xr-x 2 coder coder 2 Jul  6  2021 project
coder@fb3cb94990c3:~$ ls -l file.txr
-rw-r--r-- 1 coder coder 0 Jul 10 15:21 file.txr
coder@fb3cb94990c3:~$ cat file.txr > file2.txt
coder@fb3cb94990c3:~$ ls -l
total 2
drwxr-xr-x 2 coder coder 2 Jul 10 15:21 file1.txt
-rw-r--r-- 1 coder coder 0 Jul 10 15:22 file2.txt
-rw-r--r-- 1 coder coder 0 Jul 10 15:21 file.txr
drwxr-xr-x 2 coder coder 2 Jul  6  2021 project
coder@fb3cb94990c3:~$ ls -l file2.txt
-rw-r--r-- 1 coder coder 0 Jul 10 15:22 file2.txt
coder@fb3cb94990c3:~$ cp file.txr file2.txt
coder@fb3cb94990c3:~$ ls -l file2.txt
-rw-r--r-- 1 coder coder 0 Jul 10 15:23 file2.txt
coder@fb3cb94990c3:~$ ls -l file.txr
```

Figure 1.3 — `grep -E` matching multiple alternative patterns.

2. Viewing & Writing Files — `cat`, `ls`

Reading file contents, listing file details, and creating or updating files.

Command	Description
<code>cat .profile > file.txr</code>	Overwrites <code>file.txr</code> with the contents of <code>.profile</code> .
<code>ls -l file_name.txt</code>	Displays the file's details — permissions, owner, size, date — in long-listing format.
<code>cat file.txr >> file_name.txt</code>	Appends the contents of <code>file.txr</code> to <code>file_name.txt</code> without erasing the existing content.
<code>cat</code>	Used on its own, lets the user type lines of text directly into the terminal.
<code>cat > file_name</code>	Overwrites <code>file_name</code> with whatever text is typed immediately after the command.
<code>cat <<ADDTXT</code>	Opens a heredoc block, allowing multiple lines of text to be entered under the label <code>ADDTXT</code> .



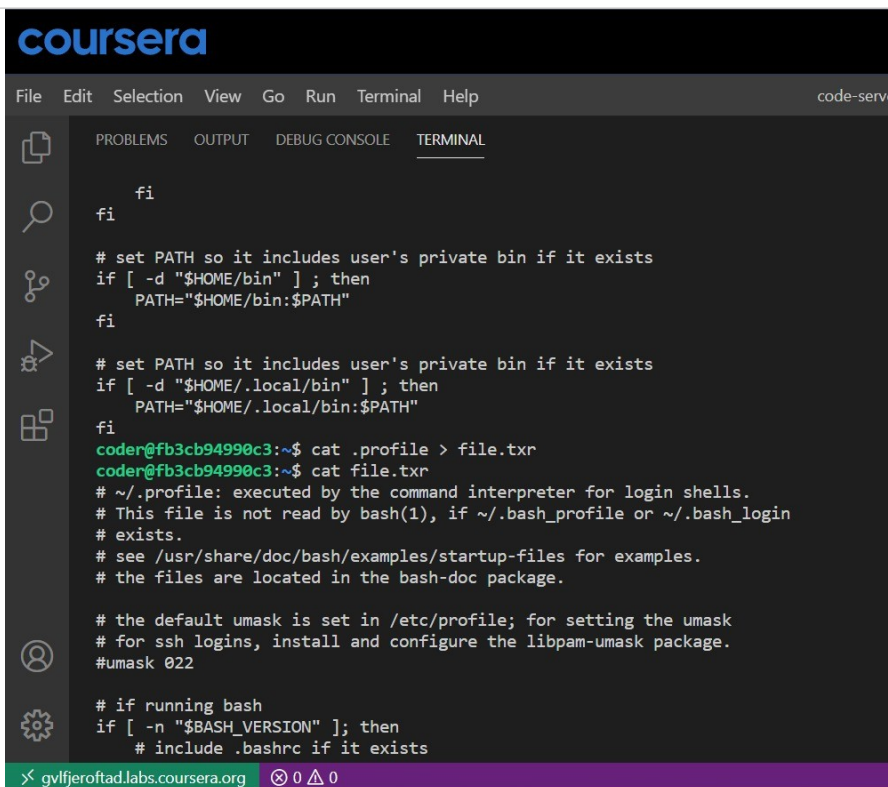
The screenshot shows a terminal window in the Coursera code editor. The terminal displays the following commands and their outputs:

```
coder@fb3cb94990c3:~$ ls -l file2.txt
-rw-r--r-- 1 coder coder 0 Jul 10 15:23 file2.txt
coder@fb3cb94990c3:~$ ls -l file.txr
-rw-r--r-- 1 coder coder 0 Jul 10 15:21 file.txr
coder@fb3cb94990c3:~$ ls file.txr
file.txr
coder@fb3cb94990c3:~$ cat file.txr
coder@fb3cb94990c3:~$ cat project
cat: project: Is a directory
coder@fb3cb94990c3:~$ ls -a
.  ..  .bash_logout  .bashrc  .config  file1.txt  file2.txt  file.txr  .local  .profile  project
coder@fb3cb94990c3:~$ cat .profile
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
# the files are located in the bash-doc package.

# the default umask is set in /etc/profile; for setting the umask
# for ssh logins, install and configure the libpam-umask package.
#umask 022

# if running bash
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
```

Figure 2.1 — Overwriting a file with cat and checking it with ls -l.



The screenshot shows a terminal window in the Coursera code editor. The terminal displays the following commands and their outputs:

```
fi
fi

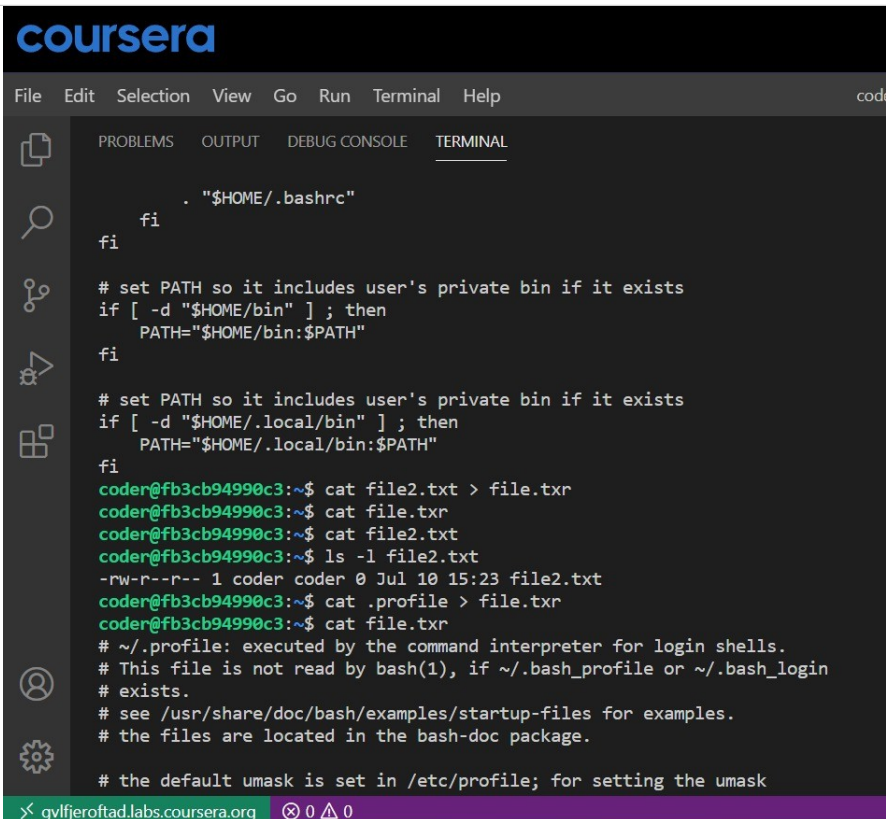
# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/.local/bin" ] ; then
    PATH="$HOME/.local/bin:$PATH"
fi
coder@fb3cb94990c3:~$ cat .profile > file.txr
coder@fb3cb94990c3:~$ cat file.txr
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
# the files are located in the bash-doc package.

# the default umask is set in /etc/profile; for setting the umask
# for ssh logins, install and configure the libpam-umask package.
#umask 022

# if running bash
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
```

Figure 2.2 — Appending content to a file with cat >>.



The screenshot shows a Coursera IDE interface with a terminal window. The terminal displays the contents of a `.bashrc` file, which includes comments and code for setting the `PATH` environment variable. The user then uses the `cat` command to append the contents of `file2.txt` to `file.txr`. The terminal output shows the file permissions and the successful execution of the `cat` command.

```
File Edit Selection View Go Run Terminal Help
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

. "$HOME/.bashrc"
fi
fi

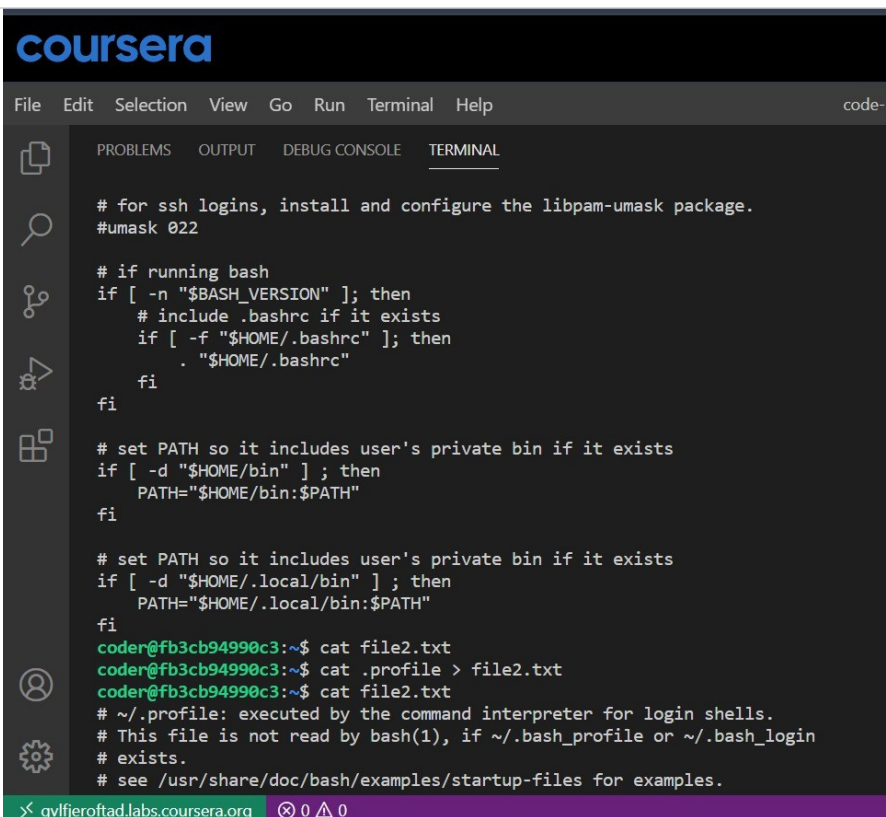
# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/.local/bin" ] ; then
    PATH="$HOME/.local/bin:$PATH"
fi

coder@fb3cb94990c3:~$ cat file2.txt > file.txr
coder@fb3cb94990c3:~$ cat file.txr
coder@fb3cb94990c3:~$ cat file2.txt
coder@fb3cb94990c3:~$ ls -l file2.txt
-rw-r--r-- 1 coder coder 0 Jul 10 15:23 file2.txt
coder@fb3cb94990c3:~$ cat .profile > file.txr
coder@fb3cb94990c3:~$ cat file.txr
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
# the files are located in the bash-doc package.

# the default umask is set in /etc/profile; for setting the umask
```

Figure 2.3 — Typing text directly into the terminal using `cat`.



The screenshot shows a Coursera IDE interface with a terminal window. The terminal displays the contents of a `.bashrc` file, which includes comments and code for setting the `PATH` environment variable. The user then uses the `cat` command to append the contents of `file2.txt` to `file2.txt`. The terminal output shows the file permissions and the successful execution of the `cat` command.

```
File Edit Selection View Go Run Terminal Help
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

# for ssh logins, install and configure the libpam-umask package.
#umask 022

# if running bash
if [ -n "$BASH_VERSION" ] ; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ] ; then
        . "$HOME/.bashrc"
    fi
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/.local/bin" ] ; then
    PATH="$HOME/.local/bin:$PATH"
fi

coder@fb3cb94990c3:~$ cat file2.txt
coder@fb3cb94990c3:~$ cat .profile > file2.txt
coder@fb3cb94990c3:~$ cat file2.txt
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
```

Figure 2.4 — Overwriting a file's content with `cat >`.

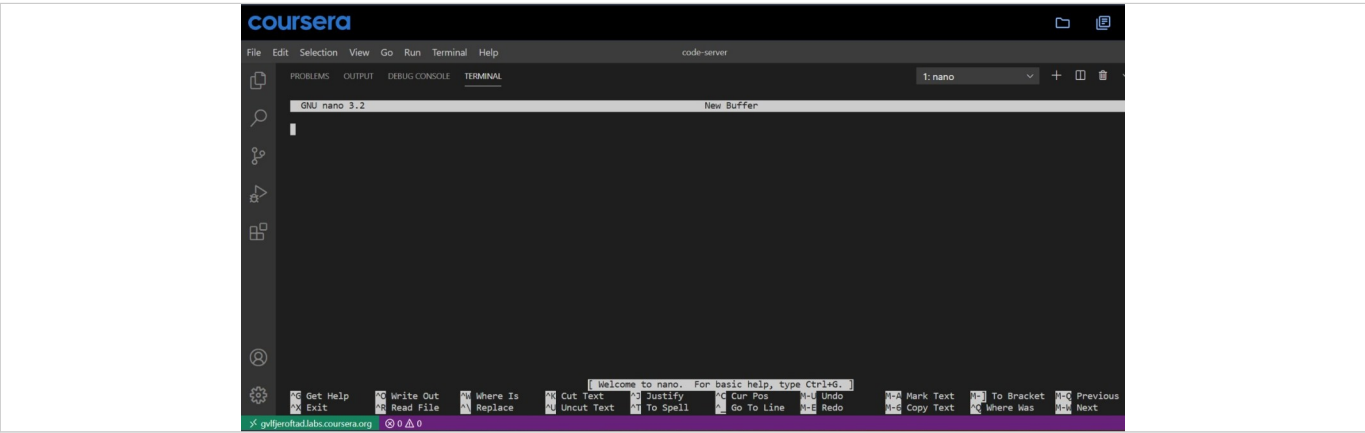


Figure 2.5 — Adding multi-line text with a cat heredoc (<<ADDTEXT).

3. File Statistics — wc

Counting lines, words, and characters, and combining results with pipes.

Command	Description
<code>wc /etc/passwd</code>	Displays the number of lines, words, and characters in /etc/passwd.
<code>wc /etc/passwd grep "passwd"</code>	Pipes () the wc output into grep to filter for lines containing “passwd”.
<code>wc /etc/passwd tee -a file_name</code>	Pipes the wc output into tee, which prints it on the terminal and simultaneously appends (-a) it to file_name.

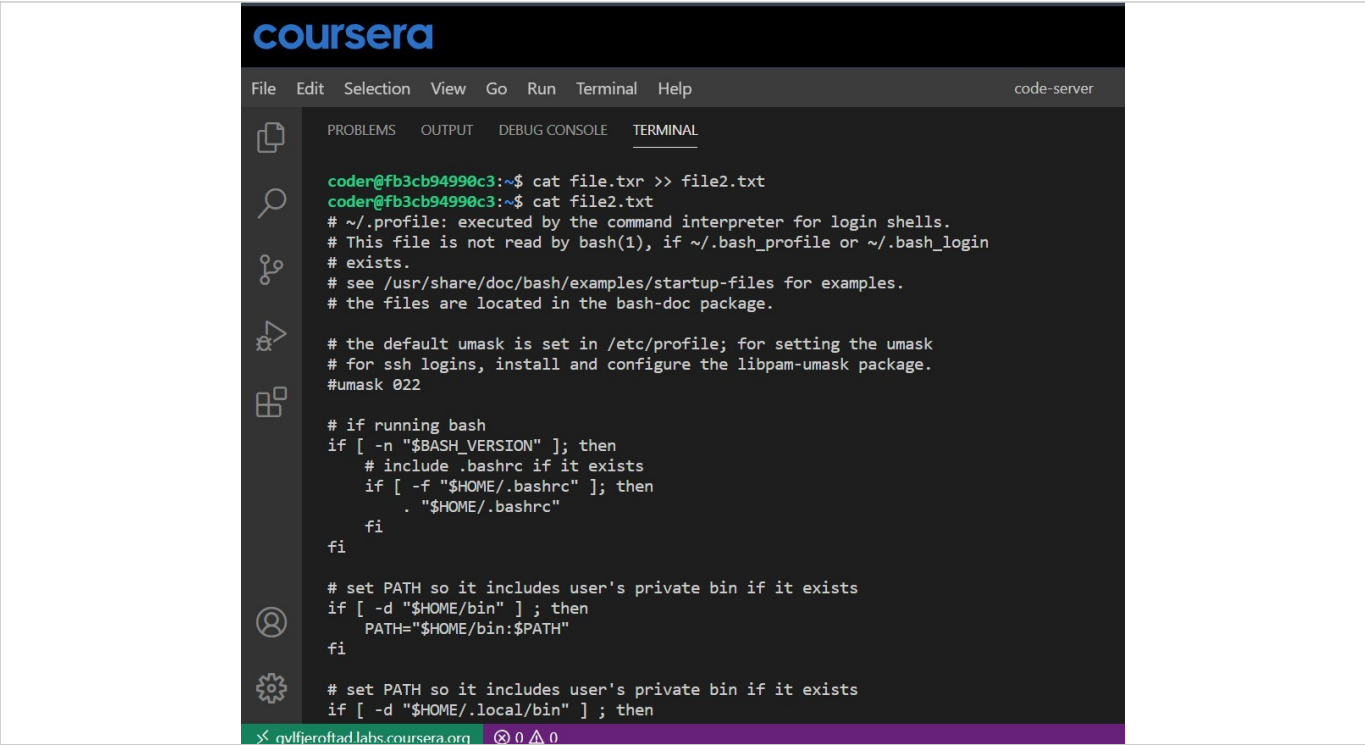
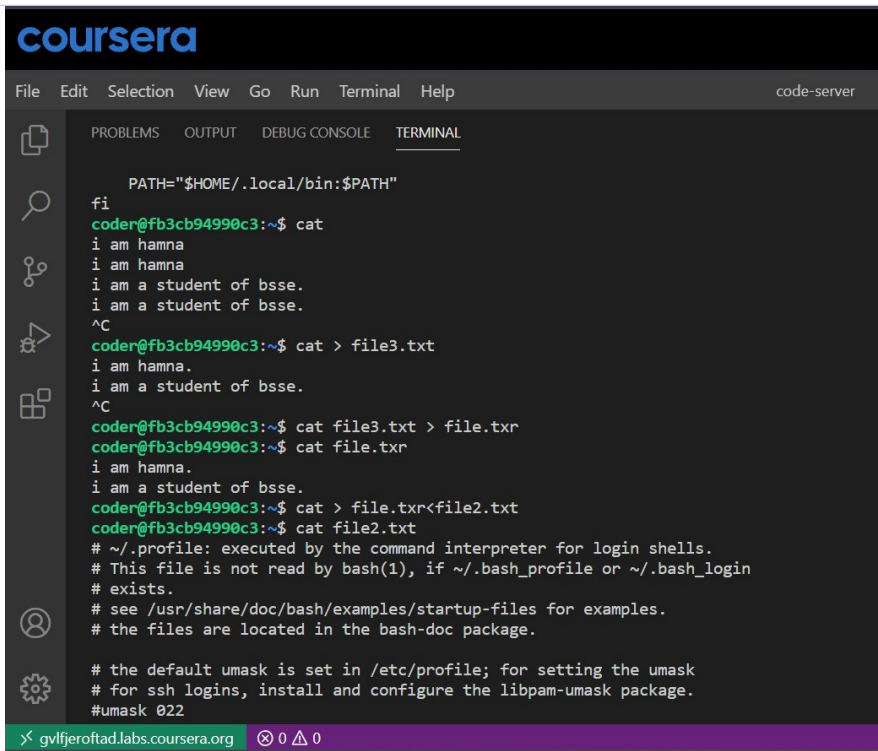


Figure 3.1 — wc showing line, word, and character counts.



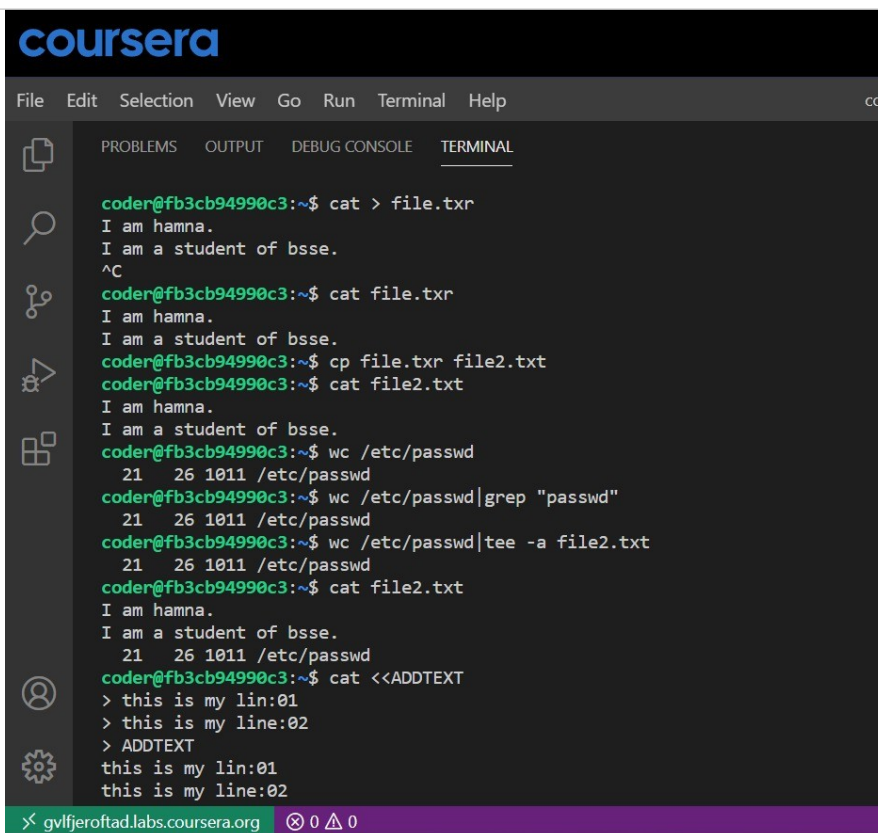
```

coursera
File Edit Selection View Go Run Terminal Help code-server
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PATH="$HOME/.local/bin:$PATH"
fi
coder@fb3cb94990c3:~$ cat
i am hamna
i am hamna
i am a student of bsse.
i am a student of bsse.
^C
coder@fb3cb94990c3:~$ cat > file3.txt
i am hamna.
i am a student of bsse.
^C
coder@fb3cb94990c3:~$ cat file3.txt > file.txr
coder@fb3cb94990c3:~$ cat file.txr
i am hamna.
i am a student of bsse.
coder@fb3cb94990c3:~$ cat > file.txr<file2.txt
coder@fb3cb94990c3:~$ cat file2.txt
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
# the files are located in the bash-doc package.

# the default umask is set in /etc/profile; for setting the umask
# for ssh logins, install and configure the libpam-umask package.
#umask 022
x gvljferoftad.labs.coursera.org 0 0

```

Figure 3.2 — Piping wc output into grep.



```

coursera
File Edit Selection View Go Run Terminal Help cod
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
coder@fb3cb94990c3:~$ cat > file.txr
I am hamna.
I am a student of bsse.
^C
coder@fb3cb94990c3:~$ cat file.txr
I am hamna.
I am a student of bsse.
coder@fb3cb94990c3:~$ cp file.txr file2.txt
coder@fb3cb94990c3:~$ cat file2.txt
I am hamna.
I am a student of bsse.
coder@fb3cb94990c3:~$ wc /etc/passwd
 21  26 1011 /etc/passwd
coder@fb3cb94990c3:~$ wc /etc/passwd|grep "passwd"
 21  26 1011 /etc/passwd
coder@fb3cb94990c3:~$ wc /etc/passwd|tee -a file2.txt
 21  26 1011 /etc/passwd
coder@fb3cb94990c3:~$ cat file2.txt
I am hamna.
I am a student of bsse.
 21  26 1011 /etc/passwd
coder@fb3cb94990c3:~$ cat <<ADDEXT
> this is my lin:01
> this is my line:02
> ADDEXT
this is my lin:01
this is my line:02
x gvljferoftad.labs.coursera.org 0 0

```

Figure 3.3 — Piping wc output into tee -a to display and append.

4. Text Editors — nano, vi

Editing files interactively from within the terminal.

Command	Description
<code>nano</code>	Opens the GNU nano text editor (version 3.2 shown) for direct in-terminal editing.
<code>-- INSERT --</code>	The nano editing mode used to type or add text; press Esc to leave this mode.
<code>:wq</code>	Saves changes and exits back to the terminal (used in vi/vim).
<code>vi</code>	Opens the vi (or vim) text editor.
<code>vi infile.txt</code>	Opens infile.txt inside vi for editing.

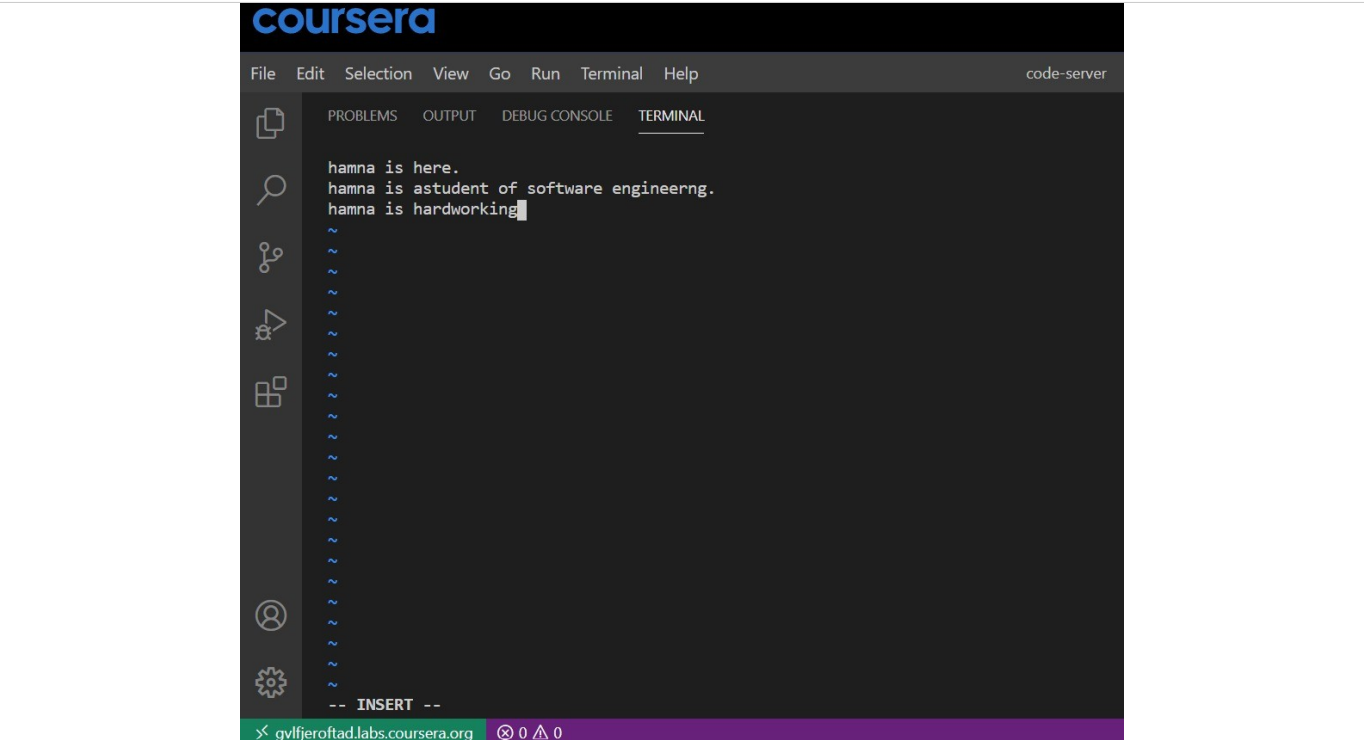
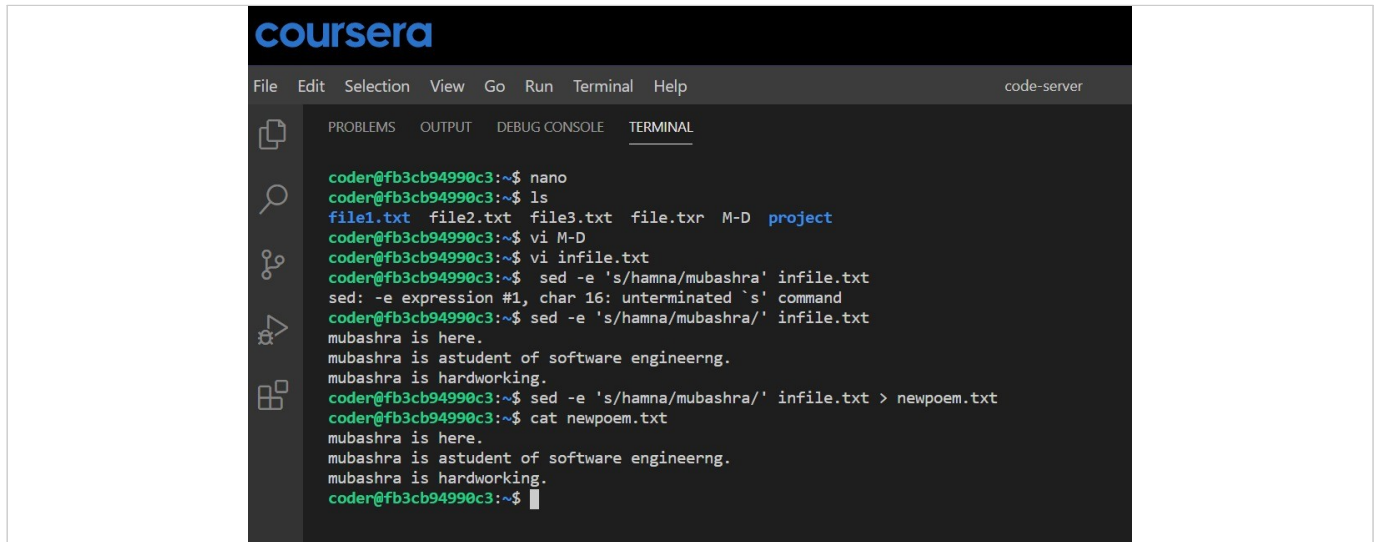


Figure 4.1 — Editing a file in nano's INSERT mode.

5. Text Processing — sed

Performing automated find-and-replace edits on file content.

Command	Description
<code>sed -e 's/hamna/mubashra/' infile.txt</code>	Replaces every occurrence of “hamna” with “mubashra” in infile.txt.
<code>sed -e 's/hamna/mubashra/' infile.txt > newpoem.txt</code>	Performs the same replacement and redirects the result into a new file, newpoem.txt.

The image shows a screenshot of a web-based code editor interface. At the top, there is a dark header with the 'coursera' logo in blue and white. Below the header is a menu bar with options: File, Edit, Selection, View, Go, Run, Terminal, and Help. On the right side of the menu bar, it says 'code-server'. The main area of the editor is divided into two panes. The left pane is a sidebar with icons for file explorer, search, and other tools. The right pane is the terminal, which has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', and 'TERMINAL'. The 'TERMINAL' tab is active, showing a command prompt session. The user is logged in as 'coder' on a machine named 'fb3cb94990c3'. The session shows the user running 'nano', listing files with 'ls', opening 'M-D' with 'vi', editing 'infile.txt' with 'vi', and using 'sed' to perform a text substitution in 'infile.txt'. The substitution replaces 'mubashra' with 'mubashra is here.' in the first line of the file. The user then saves the file and runs 'cat newpoem.txt' to view the result. The output shows the modified text.

```
coder@fb3cb94990c3:~$ nano
coder@fb3cb94990c3:~$ ls
file1.txt  file2.txt  file3.txt  file.txt  M-D  project
coder@fb3cb94990c3:~$ vi M-D
coder@fb3cb94990c3:~$ vi infile.txt
coder@fb3cb94990c3:~$ sed -e 's/hamna/mubashra' infile.txt
sed: -e expression #1, char 16: unterminated `s' command
coder@fb3cb94990c3:~$ sed -e 's/hamna/mubashra/' infile.txt
mubashra is here.
mubashra is astudent of software engineerng.
mubashra is hardworking.
coder@fb3cb94990c3:~$ sed -e 's/hamna/mubashra/' infile.txt > newpoem.txt
coder@fb3cb94990c3:~$ cat newpoem.txt
mubashra is here.
mubashra is astudent of software engineerng.
mubashra is hardworking.
coder@fb3cb94990c3:~$
```

Figure 5.1 — Using vi and sed together for text substitution.

Summary

This module walked through five essential groups of Linux command-line tools, moving from locating information inside files to editing and transforming it directly in the terminal.

- Searching & Filtering (grep) — locate lines and patterns inside files using anchors, negation, and extended regular expressions.
- Viewing & Writing Files (cat, ls) — read, overwrite, append, and inspect file content and details using redirection operators (>, >>, <<).
- File Statistics (wc) — measure line, word, and character counts, and combine results with other commands through piping (|).
- Text Editors (nano, vi) — edit files interactively from within the terminal.
- Text Processing (sed) — automate find-and-replace edits and redirect results into new files.

Together, these commands form a practical toolkit for everyday Linux work: searching for information, viewing and shaping file content, gathering quick statistics, and editing or transforming text — all directly from the command line, with pipes and redirection tying them together into efficient workflows.